lecture 1

handout every lecture
- to do list between lectures
- lecture worksheet during lecture
- questions to ask myself and to summarize/test
- practice problems
- reference sheet (all on exam)

CS106B prerequisites: C++ programming

What makes a program good?

time and space
correctness

Comparison
no standard test cases or general hardware
hard to compare universally

Solution: analytic proofs to reason about all possible inputs
formalize machine costs
asymptotic analysis regardless of hardware
analysis input on all hardware

Analyze and then formalize algorithms that are well

Sorting

Def: rearrange a list of objects in ascending order

Numbers: [11, 100, 60, 70] → [11, 60, 70, 100]
Words: [word, worm, work] → [word, work, worm]

Why: - common
- historical
- pedagogical: good problem to practice reasoning

Selection Sort Code

```
1  def selection_sort(A, N):
2      for i ∈ 0, 1, ..., N-1:
3          m ∈ i                          value for smallest     | unit
4          for j ∈ i+1, i+2, N-1:
5              if A[m] > A[j]:            | unit
6                  m ∈ j                  | unit
7          swap A[i] ↔ A[m]               | unit
8      return A                           | unit
```

Correctness: later
time
assumptions: each code line takes 1 unit of time

[17, 9, 15]

Experiment
Pick input of length n
run selection sort
count iterations

best case: ascending
worst case: descending

line 5:

i	time units
0	(N-1) - (0+1) + 1 = N-1
1	(N-1) - (1+1) + 1 = N-2
⋮	
N-1	(N-1) - (N-1+1) + 1 = 0

(N-1) + (N-2) + ... + 1 + 0 = $\sum_{i=1}^{N-1} i$ = $\frac{1}{2}N^2 - \frac{1}{2}N$

0
1
2

Compare best and worst

lower: $0.5n^2 + 1.5n + 1$
upper: $n^2 + n + 1$

Compare the distributions at the extremes

Locality factor does have an impact

June 22nd lecture Quiz.

N-2 outer loop

the last iteration does not need a check. It is already sorted

2598

5298
2598